2019 IEEE Intelligent Vehicles Symposium (IV)
Paris, France. June 9-12, 2019

Controlling an Autonomous Vehicle with Deep Reinforcement Learning*

Andreas Folkers[1] Matthias Rick[1] Christof Büskens[1]

***Abstract*— We present a control approach for autonomous vehicles based on deep reinforcement learning. A neural network agent is trained to map its estimated state to acceleration and steering commands given the objective of reaching a specific target state while considering detected obstacles. Learning is performed using state-of-the-art proximal policy optimization in combination with a simulated environment. Training from scratch takes five to nine hours. The resulting agent is evaluated within simulation and subsequently applied to control a full-size research vehicle. For this, the autonomous exploration of a parking lot is considered, including turning maneuvers and obstacle avoidance. Altogether, this work is among the first examples to successfully apply deep reinforcement learning to a real vehicle.**

I. INTRODUCTION

Self driving cars have the potential to sustainably change modern societies which are heavily based on mobility. The benefits of such a technology range from self-providing car sharing to platooning approaches, which ultimately yield a much more effective usage of vehicles and roads [1]. In recent years, great progress has been made in the development of these systems, where the results achieved by deep learning methods have been a major factor. One example is the neural network PILOTNET, which was trained to steer a vehicle solely based on camera images [2].

More classic methods divide the processing of sensor data and the calculation of vehicle controls into separate tasks. The latter can be achieved by various model-based control approaches, one method being the linear quadratic controller [3], also known as Riccati controller. It minimizes a quadratic objective function in the state deviation and control energy while taking into account a linear model of the underlying system. There are various examples for the application of this technique to autonomous driving [4], [5], [6].

While a Riccati-controller is comparably fast, however, it does not directly allow the consideration of constraints such as obstacles or more advanced objective functions within the optimization. Such requirements are met by a general nonlinear model predictive control (MPC) approach based on solving an optimal control problem at every time step. Although the calculations required are considerably more complex, such methods were successfully implemented for autonomous vehicles, e. g., [7] or most recently [8] utilizing efficient solvers such as TRANSWORHP [9].

*The authors would like to thank the Federal Ministry for Economic Affairs and Energy of Germany (BMWi) and the German Aerospace Center (DLR) Space Administration for supporting this work [grant nos. 50NA1615 and 50NA1909].

[1]WG Optimization and Optimal Control, Center for Industrial Mathematics, University of Bremen, Germany afolkers@uni-bremen.de

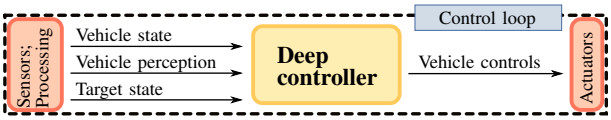


Fig. 1: Control loop considered in this work.

A combination of the advantages of both, the speed of the Riccati controller and the generality of MPC, can be achieved by finding a function that maps state values to control variables, e. g., by training a deep neural network. Such a model could, for example, be learned supervised, as done for PILOTNET, or by reinforcement learning. The latter in particular led to excellent results in the training of such agents for controlling real-world systems such as robots [10] or helicopters [11].

Recent work also shows promising applications of reinforcement learning for autonomous driving by making strategic decisions [12], [13] or by the computation of control commands [14], [15], [16]. Although these results show the success of this approach in simulated environments, there are very few examples of evaluations on real vehicles. One of them was presented by the WAYVE research team where a policy for lane following is learned based on corresponding camera images [17]. Training is done online, with the only feedback for improvement coming from the intervention of a safety driver. While this methodology works when training is carried out without the proximity of real obstacles and at low speeds, it may be difficult to implement this approach for more general situations.

This work extends the results described previously by realizing the autonomous exploration of real a parking lot based on deep reinforcement learning. A key contribution is the definition of a suitable Markov decision process which allows fast training and leads to agents capable of being deployed on a full-size vehicle. While the data generation during learning is based on an efficient model-based simulation, the policies are designed as neural networks and trained by a continuous proximal policy optimization algorithm [18]. Finally, this results in a *deep controller* which is able to provide sophisticated control commands in a fixed computation time of approximately 1.2 ms. The corresponding control loop is based on the estimated state and perception of the vehicle as well as a target state, as depicted in Fig. 1. The performance of the learned controller is evaluated both in simulation and in real-world experiments. Since the exploration of a parking lot involves sharp turning maneuvers and road constrictions caused by obstacles, it can be considered significantly more challenging than mere lane

978-1-7281-0560-4/19/$31.00 ©2019 IEEE

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:21:09 UTC from IEEE Xplore. Restrictions apply.

following. Hence, to the best of our knowledge, this work extends the state-of-the-art results for successfully driving a real autonomous vehicle based on a deep reinforcement learning policy.

II. CONTINUOUS DEEP REINFORCEMENT LEARNING

In recent years, various methods from classical reinforcement learning [19] have been combined with neural networks and their optimization through backpropagation [20], leading to algorithms such as Deep Q-Learning [21], and several actor-critic-methods [16], [22], [23]. In particular, the latter class of algorithms allows agents to be trained with continuous action spaces, which is crucial for their application as a controller on a real-world system like an autonomous vehicle. Our training procedure is based on the proximal policy optimization (PPO) algorithm [18]. For this, an infinite Markov decision process (MDP) is considered, defined by the six-tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}_{sa}^{s'}, \mathcal{P}_0, r, \gamma)$, with $\mathcal{S}$ and $\mathcal{A}$ being the continuous and bounded spaces of states and actions, respectively. The probability density function $\mathcal{P}_{sa}^{s'}$ characterizes the transition from state $s \in \mathcal{S}$ to $s' \in \mathcal{S}$ given action $a \in \mathcal{A}$ while $\mathcal{P}_0$ incorporates a separate distribution of possible start states. The reward function is denoted by $r : \mathcal{S} \times \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ and γ is the discount factor.

With these definitions the goal of a reinforcement learning algorithm is to find an optimal policy $\pi : \mathcal{S} \times \mathcal{A} \to [0, 1]$ representing the probability density of the agent's action, given a certain state. Optimality is specified in relation to the expected discounted return

$$\begin{aligned} &\eta(\pi) := \mathbb{E}_{s_0, a_0, \ldots} \left\{ \sum_{t=0}^{\infty} \gamma^t r_{t+1} \right\}, \\ &s_0 \sim \mathcal{P}_0(s_0), \quad a_t \sim \pi(a_t|s_t), \quad s_{t+1} \sim \mathcal{P}_{s_t a_t}^{s_{t+1}}, \end{aligned} \tag{1}$$

with $r_{t+1} := r(s_{t+1}, s_t, a_t)$. While in the actor-critic setting the policy π is identified with the actor, the state-value function

$$V^{\pi}(s_t) := \mathbb{E}_{a_t, s_{t+1}} \left\{ \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \right\}$$

can behave as the critic which is responsible to evaluate the policy's actions. Correspondingly, the state-action-value function and the advantage are given by

$$\begin{aligned} &Q^{\pi}(s_t, a_t) := \mathbb{E}_{s_{t+1}, a_{t+1}} \left\{ \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \right\} \quad \text{and} \\ &A^{\pi}(s_t, a_t) := Q^{\pi}(s_t, a_t) - V^{\pi}(s_t). \end{aligned}$$

To find a (nearly) optimal policy in continuous state and action spaces, it proved helpful to use neural networks as function approximators, which leads to π_θ and V^θ being parameterized. Altogether, a deep actor-critic training algorithm has to find parameters θ to both maximize the true target (1) and approximate the corresponding state-value function V^θ.

Regarding the former, proximal policy optimization is based on a first order approximation of η around a reference policy π_{θ_0} for the local optimization of the parameters of π_θ. The distance between both is approximately measured by

$$\xi_t^{\theta_0}(\theta) := \tfrac{\pi_\theta(a_t|s_t)}{\pi_{\theta_0}(a_t|s_t)}.$$

The samples (s_t, a_t) to be evaluated are generated within the computation of a rollout set $\mathcal{M}$. For this purpose a total of N data points is computed by following the reference policy π_{θ_0}. If a terminating state is reached in episodic tasks, a new start state is sampled with respect to $\mathcal{P}_0$. Altogether, the PPO objective for training the policy π_θ is given by

$$\begin{aligned} \zeta_\pi(\theta) := -\mathbb{E}_{(s_t, a_t) \in \mathcal{M}} \Big(&\min \big[\xi_t^{\theta_0}(\theta) \hat{A}_t, \\ &\operatorname{clip}(\xi_t^{\theta_0}(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \big] \Big) \end{aligned} \tag{2}$$

which introduces a pessimistic balancing of two terms controlled by the clip parameter $\varepsilon > 0$ [18]. The advantage of the tth data point is approximated by $\hat{A}_t$ which can further be used to define a cost function ζ_V for V^θ as the quadratic error

$$\zeta_V(\theta) := -(\hat{A}_t)^2. \tag{3}$$

Finally, a robust approximation $\hat{A}_t$ can be computed using the generalized advantage estimation [24] given as

$$\hat{A}_t^{\gamma,\lambda} := \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}^{\theta},$$

which allows a sophisticated trade-off between bias and variance through the parameter $\lambda > 0$. Since the rollout set $\mathcal{M}$ is given by trajectories, this estimate can be computed for each data point by summing up to the end of the corresponding episode. The implementation of the PPO method is summarized in Algorithm 1.

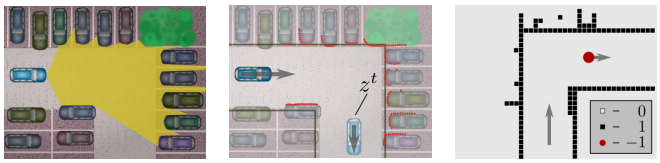
Algorithm 1: Proximal policy optimization

```
Initialize π_θ and V^θ;
while not converged do
    Set π_{θ_0} ← π_θ;
    Generate a rollout set M following π_{θ_0} and
     compute Â_t^{γ,λ} for each data point;
    for K steps do
        Draw a random batch of size M from M;
        Update π_θ and V^θ using a stochastic gradient
         descent algorithm with backpropagation and
         the cost functions ζ_π (2) and ζ_V (3);
    end
end
```

III. DEEP CONTROLLER FOR AUTONOMOUS DRIVING

This work is part of the research project AO-CAR whose objective is the development of algorithms for navigation and optimal control of autonomous vehicles in an urban environment [8]. Within the setting of the project, information about the vehicle's surrounding are measured by, e. g., laser scanners and are further extended by a rough knowledge about the geometry of the drivable area (see Fig. 2). Based on this, a desired target state z^t is defined, including a speed value, as described in [25] and illustrated in Fig. 2b for the exemplary situation of the autonomous parking lot

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:21:09 UTC from IEEE Xplore. Restrictions apply.



(a) Sensors. (b) Target state z^t. (c) Perception $\mathcal{O}$.

Fig. 2: During the exploration of a real parking lot, obstacles are perceived by sensors (a). Together with the knowledge about the drivable area, a current target state is defined (b). The surrounding from the perspective of the vehicle can be described by a ternary perception map. Here, the target is represented by a red dot (c).

exploration. The measurements and targets are updated at high frequency, ultimately resulting in a control loop. The task of the corresponding controller is to provide steering and acceleration values at every iteration, so that a safe trajectory to the current target is obtained. Within this setting, all other obstacles are assumed to be static.

We demonstrate how a deep reinforcement learning agent is trained to implement a controller for solving this task. In particular, this includes the definition of a suitable MDP as well as, in accordance to it, the design of neural networks topologies of π_θ and V^θ. Finally, the network parameters can be learned as described in Algorithm 1 within an appropriate environment. Here, in contrast to [17], training is performed based on a simulation in order to quickly obtain a model as general as possible from various and diverse situations. Within the resulting episodic MDP, the policy has to make the vehicle reach the desired target state including a specified speed value. In particular, this should lead to a controlled stop, if the latter is zero. After this training step, the resulting agent can be deployed to control a real-world vehicle. Details of the simulated MDP are presented in the following.

A. State and Action Spaces

The research vehicle can be controlled algorithmically by specifying the steering wheel angle ν as well as the longitudinal acceleration a. The former can be mapped bijectively to the mean angle of the front wheels, defined as β. To prevent arbitrary fast changes, the angular velocity $\omega = \dot{\beta}$ is defined as part of the action space instead of β or ν, leading to

$$\mathcal{A} := \{(a, \omega)^\top \in \mathbb{R}^2 \mid \text{both bounded}\}.$$

These control variables have direct influence on the set of vehicle coordinates

$$\mathcal{Z} := \{(x, y, v, \varrho, \beta)^\top \in \mathbb{R}^5 \mid v, \varrho, \beta \,\text{bounded}\}$$

as shown in Fig. 3. The tuple (x, y) describes the position of the center of the vehicle's rear axle with respect to an inertial system. The speed in the longitudinal direction is called v, where $\dot{v} = a$, and the orientation of the vehicle with respect to the inertial system is referred to as ϱ.

Both, position and orientation of the current target $z^t \in \mathcal{Z}$, can be expressed by the bounded relative position

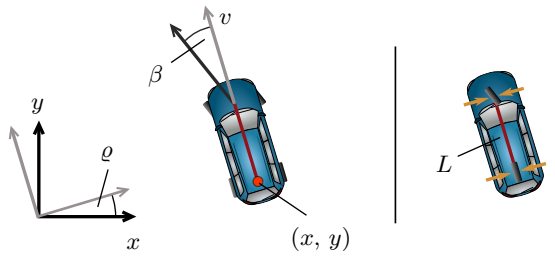


Fig. 3: Left: Coordinates of the vehicle state vector. Right: Single track model simplification.

$p^r := (x^r, y^r)$ with respect to the vehicle coordinates $z \in \mathcal{Z}$ and the complex number representation $(\varrho^r_\Re, \varrho^r_\Im)$ of the corresponding relative orientation. The form of the latter has the advantage of avoiding discontinuities. Moreover the controller needs to know the desired target speed as well as the current speed of the vehicle in order to predict its next states and to allow safe driving maneuvers. The target steering angle β^t is not of interest, which ultimately leads to the first part of the state space

$$\tilde{\mathcal{S}} = \{(x^r, y^r, v, v^t, \beta, \varrho^r_\Re, \varrho^r_\Im)^\top \in \mathbb{R}^7 \mid \text{all bounded}\}.$$

The second component is given by the vehicle's relative perception of its surrounding. This can be described by a ternary obstacle grid $\mathcal{O}$ of size $n \times m$ with entries in $\{-1, 0, 1\}$, as shown in Fig. 2c. In it, free space is encoded by 0's, while obstacles, measured through sensors, as well as lane boundaries are indicated by 1's. Furthermore the target position is given as -1. Finally the MDP state space is defined as

$$\mathcal{S} := \{(\tilde{s}, \mathcal{O}) \mid \tilde{s} \in \tilde{\mathcal{S}}, \mathcal{O} \in \{-1, 0, 1\}^{n \times m}\}.$$

B. State Transitions

The action-dependent transition between states of a real vehicle can be incorporated within the simulation using a system of differential equations to describe its behavior. This leads to an update of the relative vehicle coordinates as well as a new measurement of the obstacle perception. For low speeds on a parking lot, kinematic considerations such as simple single-track-models are sufficient [26]. Here, the vehicle is assumed to have only one wheel at the front and back respectively, each centered between the real ones as shown in Fig. 3. This leads to the system of equations

$$\left(\dot{x}, \dot{y}, \dot{v}, \dot{\varrho}, \dot{\beta}\right)^\top = \left(v\cos(\varrho), v\sin(\varrho), a, {}^{v}/_{L}\tan(\beta), \omega\right)^\top,$$

where L is the vehicle-specific wheelbase [27]. Further physical constraints, such as a limited steering angle, can be directly considered within the simulation.

C. Reward Function

During learning in simulation, the reward function r encodes the agent's goal to reach fixed target positions and orientations as comfortable and safe as possible while maintaining the corresponding desired speed. However, for the training stage to be successful, two main difficulties must be considered when defining the reward.

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:21:09 UTC from IEEE Xplore. Restrictions apply.

First, due to an obstacle blocking the lane or because it is demanded by the human user, the agent must also be able to stop at a specified position. In this case, it is desired to maintain a comparably slow speed and perform a controlled stop as soon as the target is reached. Since driving and stopping maneuvers require a rather different behavior of the agent, two separate policies, the DRIVER and the STOPPER, are learned based on corresponding rewards r^D and r^S.

Second, since the present MDP is continuous, it is almost impossible to ever fulfill the given tasks starting with a random policy and to learn from that success. As a result, training is performed in two phases with corresponding rewards $r_1^{\{D,S\}}$ and $r_2^{\{D,S\}}$. For both policies, the first learning phase rewards proximity at every time step of the episode. Reaching the final position or rather performing a stop is rewarded specially. For the exemplary case of the DRIVER this results in

$$r_1^D := c_p(p^\curlyvee)^2 + [r_0]_p + [1/2 r_0]_{p,\varrho}, \quad c_p, r_0 \in \mathbb{R}_+,$$
$$p^\curlyvee := \max\left\{1 - \frac{\|p^r\|_2}{p_{\max}}, 0\right\} \tag{4}$$

where $p^\curlyvee$ measures the proximity to the target and $[\cdot]_p$ (or $[\cdot]_{p,\varrho}$) is only granted if the agent reaches the desired position (or position and orientation respectively). As soon as the policy learns to get high rewards in the first phase, additional behaviors such as an appropriate speed and small steering angles are taken into account. This is achieved by

$$r_2^D := r_1^D + c_v v^\curlyvee + c_\beta \beta^\curlyvee, \quad c_v, c_\beta \in \mathbb{R}_+,$$

in case of the DRIVER. Again, $v^\curlyvee$ and $\beta^\curlyvee$ measure the proximity to the respective desired speed and steering values, as illustrated in (4). To learn the STOPPER policy, similar rewards apply and are complemented with, e. g., an additional weight on the desired speed in the first phase.

D. Policy and Value Function as Neural Networks

As suggested in Sect. II, two neural networks are trained to be identified as policy π_θ and value function V^θ. Here, both share the same topology but not the same parameters. The state component $\tilde{s} \in \tilde{\mathcal{S}}$ is processed by two dense layers of each 200 ReLU activations, as shown in Fig. 4. On the other hand, the evaluation of the perception map $\mathcal{O}$ is based on two convolutions, which allow to learn from the structure of the input. The spatial dimension is halved by a max pooling operation both times. While the first convolution consists of $C = 30$ feature maps $\{\Sigma_c\}_{c=1,\dots,C}$, the latter is reduced to only one, which is then flattened and also processed by a dense layer of 200 ReLU activations. The result of both inputs is then concatenated and passed through a last 200 ReLU layer.

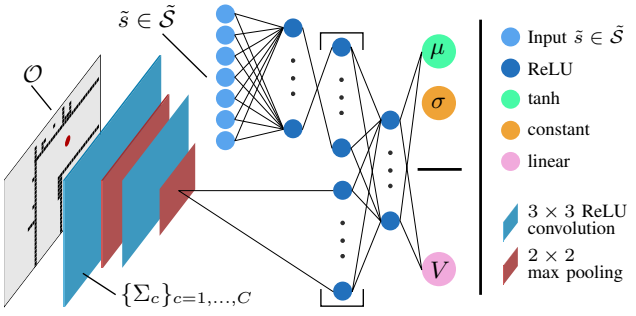


Fig. 4: Neural network topology of policy π_θ (top) and value function V^θ (bottom).

Finally, the output V of the value function model V^θ is computed by a subsequent dense layer of one linear activation. Furthermore, the return of the policy π_θ is a pair (μ, σ) for each possible action, which is identified with the mean and standard deviation of a gaussian probability distribution. While the former is computed based on a $\tanh$ activation function, the latter is defined to be independent from the input, which yields a general measure of how certain the model is about its actions. In particular, the noise introduced by σ controls the policy's level of exploration when defining the rollout set $\mathcal{M}$. Furthermore, it results in robustness against disturbances in the expected state transition with respect to the most preferred action μ.

Constraints on the control commands can be incorporated by properly scaling the $\tanh$ activation and an additional clip operation in the case of action selection based on the normal distribution.

IV. TRAINING AND RESULTS

We evaluate our training procedure and present the performance of the resulting policy in simulation and on a real vehicle. Learning is realized on a GTX 1080 GPU while the simulator is conducted by an Intel Xeon E5 CPU kernel. Within the latter, random control tasks are generated as shown in Fig. 5. The reward is shaped to make the policy drive the vehicle to the target state, which would define the end of an episode. Alternative termination criteria are the collision with an obstacle or the boundary polygon, exceeding a speed value of $3.3\,\mathrm{m/s}$ $(12\,\mathrm{km/h})$ as well as reaching the maximum time step $T = 250$. The simulated time between

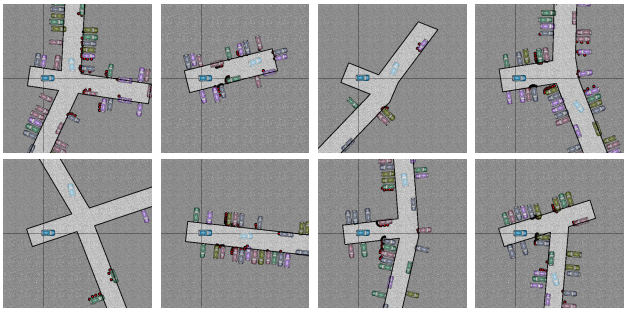


Fig. 5: Randomly selected scenarios computed by the simulator. The initial state is represented by the dark blue vehicle at the center of the coordinate system and includes random values for speed v and steering angle β. The light blue car defines the target. Red dots indicate sensor measurements of obstacle vehicles and the drivable area is defined by a closed polygon.

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:21:09 UTC from IEEE Xplore. Restrictions apply.

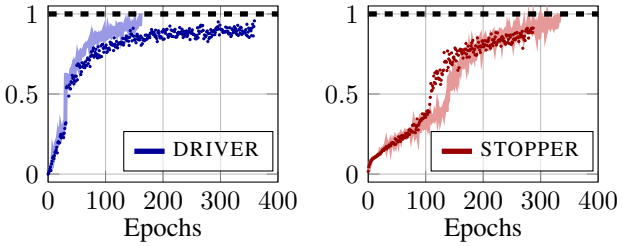


Fig. 6: Normalized average reward during learning of the DRIVER and STOPPER policy. The dotted lines represent training with obstacle vehicles (type A), while the solid lines indicates that they were excluded (type B). Progress is displayed until the respective maximum value is reached.

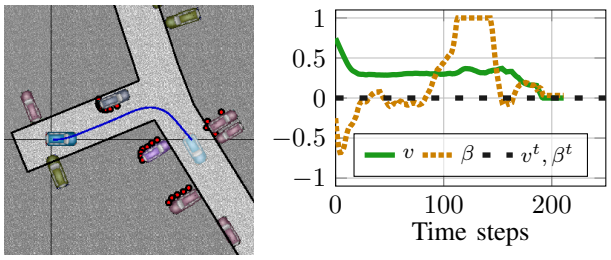


Fig. 7: Exemplary solution of a STOPPER agent which was trained without obstacle vehicles (type B). The values in the right plot are normalized to their respective maximum.

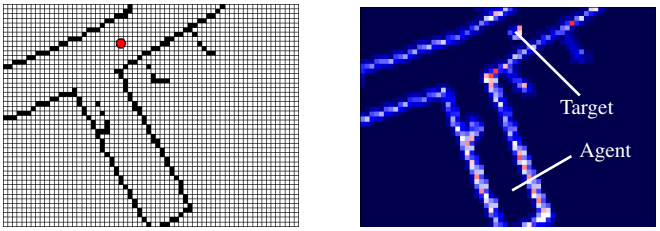


Fig. 8: Perception input $\mathcal{O}$ (left) and its attention map $\mathbb{A}$ (right) of the initial situation in Fig. 7. While dark blue pixels indicate lowest attention, it is at maximum for red ones.

two such steps is defined to be 100 ms. The control values a and ω are bounded by $\pm 1.2\,\mathrm{m/s^2}$ and $\pm 1.2\,\mathrm{rad/s}$ respectively.

Proximal policy optimization is carried out by collecting rollout sets consisting of 16,384 time steps, which are then used within $K = 16$ optimization steps with batches of size $M = 1{,}024$. One such epoch takes on average 150 s, while a maximum of 350 epochs is required for convergence. This results in a training time of less than 15 h. However, policies of similar quality can be obtained even without incorporating other vehicles as obstacles during the learning process, as outlined in the next section. The resulting simulation is much faster without detailed computation of the agent's perception, which leads to optimization cycles of 90 s. This reduces the maximum training time to 9 h.

The typical development of the average reward during training is shown in Fig. 6. As one would suggest, convergence of a STOPPER policy is much slower in the beginning, due to the fact that it is harder for the agent to learn about its final goal. DRIVER models usually converge in less than 200 epochs in the simplified setting, which results in a computation time of 5 h. Furthermore, the overall rewards are considerably lower in the case of obstacle vehicles included during learning. Since that task is more difficult, this result is also to be expected.

A. Results within the Simulation

We evaluate DRIVER and STOPPER models resulting from learning with obstacle vehicles (type A) and without them (type B), while all other parameters remain identical. In particular, we demonstrate their performance within the simulated environment which is summarized in Table I. The evaluation is done based on 10,000 randomly generated control tasks including other vehicles. Results show high success rates with other terminations mainly due to obstacles. Further investigations show that this outcome is very often caused by an infeasible combination of initial orientation, speed and steering angle with respect to the placement of obstacles which would trigger an emergency brake in reality. Even though type B models are only trained based on the polygon representing the drivable area, they still achieve results of similar quality than the type A agent. In case of the DRIVER model they are even better. Most importantly this indicates that type B models are capable of handling unknown obstacles, even if these lead to constrictions on the lane.

TABLE I: Comparison of termination criteria and average reward of models trained with (type A) and without obstacles (type B). Values are generated in 10,000 simulation runs including other vehicles. The average cumulated reward per episode is normalized to their respective maximum.

Policy	Obstacle Training		∅ Rew.	Termination [%] Succ.	Coll.	Time	Speed
DRIVER	✓	(A)	1.0	90.7	6.1	3.2	0.0
	-	(B)	0.984	87.5	10.5	2.0	0.0
STOPPER	✓	(A)	0.967	80.8	11.5	7.8	0.0
	-	(B)	1.0	82.9	14.5	2.6	0.0

This result is further supported by Fig. 7 and Fig. 8 which present an exemplary task solved by a type B STOPPER policy. The former displays the path as well as the corresponding normalized speed and steering values when applying this agent. In particular, the agent is not pulling out before turning to take the obstacle vehicle on the lane into account. Fig. 8 shows the perception map $\mathcal{O}$ of the initial situation in Fig. 7 relative to the agent. To evaluate what parts of it are most important to the neural network, an *attention map* $\mathbb{A}$ can be computed as suggested in [28]. For that the feature maps $\{\Sigma_c\}_{c=1,\dots,C}$ of the first convolutional filter (see Fig. 4) are squared and summed up, yielding $\mathbb{A} = \sum_{c=1}^{C} \Sigma_c^2$. In the present situation, high attention is paid to the agent's immediate surrounding as well as to important points in the long term such as the corner, where it has to turn around, or the surrounding of the target. In addition, comparably high attention is paid to the obstacle standing on the lane in front

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:21:09 UTC from IEEE Xplore. Restrictions apply.

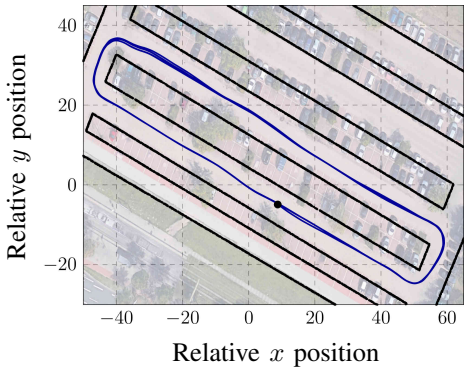


Fig. 9: Exemplary path (blue line) taken by the deep controller when five times circling a real parking area counterclockwise. The positions are relative to the prior knowledge about the parking lot (black lines). The start is marked by a black dot. The underlying satellite image (© 2009 GeoBasis-DE/BKG) is inserted as a reference only (so it does not show the actual occupancy of parking spaces).

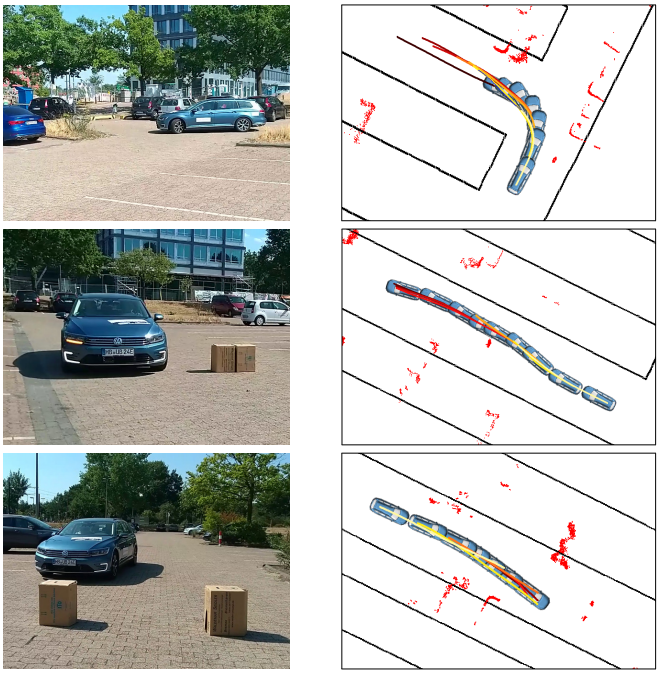


Fig. 10: Application of the deep controller on a real vehicle in three exemplary scenarios. Red dots indicate laser measurements and the drivable area is marked by black lines. The currently planned path of the controller is given by trajectories with a color gradient from light yellow to dark red. Top: Turning. Middle: Driving around an obstacle and stopping afterwards. Bottom: Detection of a blockade on the street and stopping in front of it.

of it. Altogether, these results indicate that a high quality agent can be obtained even with a simple simulation in short training time.

B. Results on a Real Vehicle

After the training in simulation has been completed, a STOPPER and DRIVER model are combined into one deep controller. This is applied within the system for autonomous driving, developed as part of the research project AO-Car [8], to be the main controller during the autonomous exploration of a parking lot (as described in Sect. III). Experiments are performed on a standard Volkswagen Passat GTE Plug-in-Hybrid with additional laser scanners at its front and rear.

During exploration, an estimate of the vehicle's state is computed every 20 ms using an Extended Kalman Filter as described in [25]. From this, a current target state is deduced and corresponding control commands are computed by the controller. Due to the target being updated at high frequency, the vehicle continues driving until a position to stop at is provided. Otherwise, the target speed is set to $8\,\mathrm{km/h}$. Note that speed values of up to $12\,\mathrm{km/h}$ were learned - driving faster can, however, be incorporated into the training process by adjusting the corresponding termination value (see also the introduction to Sect. IV). In this case, depending on the speed requirements, it may be beneficial to use a more accurate vehicle model than the one presented in Sect. III-B.

Since the outputs of the deep controller are the acceleration and the steering angle velocity, the latter is numerically integrated for another 20 ms yielding the desired steering angle and thus ultimately the vehicle control. The computation of one control command by the neural network representing the policy takes approximately 1.2 ms on an Intel Core i7-4790.

General characteristics of the deep controller applied to a real vehicle can be assessed on the basis of Fig. 9. One can see that the driven path is in the center of the lane at all times, leading to a high safety of the corresponding trajectory. This applies in particular to the turning maneuvers, after which the vehicle is immediately aligned with the lane again. One can further notice that the paths taken in every turn are very similar to each other. The only deviations occur at the start or in the case of a third party vehicle influencing the control commands (top left).

Fig. 10 shows three specific scenarios which further demonstrate the performance of the proposed controller. In particular, we overlaid a series of state estimates and the corresponding trajectories that the agent would execute based on the vehicle's current perception. One can see, again, that during the turning maneuver the application of the controller leads to a maximized safety margin to all obstacles. In particular, this allows a safe passing of the bush (next to the parking blue car), which is not entirely captured by the sensors of the research vehicle. In addition, the system is able to make decisions based on newly detected objects. On request, the deep controller is able to execute a safe evasive maneuver or to perform a soft stop.

In summary, applying the deep controller leads to a very safe and pleasant driving behavior of the research vehicle in all cases. Hence, we have shown that the presented method is capable of providing sophisticated control commands and at the same time fulfilling strict requirements in terms of computation time.

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:21:09 UTC from IEEE Xplore. Restrictions apply.

V. CONCLUSION

We presented a general design approach for a nonlinear controller that is able to provide highly advanced control commands in an extremely short computation time of 1.2 ms. This was done by approximating the control problem within a simulation as a Markov decision process which was then solved by an agent in the setting of a reinforcement learning problem. Training was performed by a proximal policy optimization method with the policy being defined as a neural network. We evaluated our approach in the context of autonomous driving and showed that a high quality controller could be obtained within a few hours of training. Furthermore, we demonstrated its performance on a full-size research vehicle during the autonomous exploration of a parking lot. For example, the controller was able to handle sharp turnings as well as new obstacles by performing an evasive maneuver or a stop. In particular, this work is one of the first successful and, to the best of our knowledge, the currently most general application of a deep reinforcement learning agent to a real autonomous vehicle. Future work will include a more detailed analysis of the neural network structure and the state representation as well as applications to further scenarios that are part of the AO-CAR follow-up project OPA^{3}L. The training process as well as the evaluation on the research vehicle are available as a video at `https://youtu.be/1HwHdL7bY3A`.

APPENDIX - HYPERPARAMETERS

TRAINING

Descript.	Var.	Value
Steps / episode	T	250
Step length		0.1 [s]
Rollout size	N	16,384
Optim. / rollout	K	16
Minibatch size	M	1,024
Discount	γ	0.99
Gener. advantage	λ	0.95
clip parameter	ε	0.1
Scaling of ζ_V		0.1
Adam optimizer	α	5e−5
(cf. [29])	β_1	0.9
	β_2	0.999
	ϵ	1e−5

REWARD

Descript.	Var.	Value
DRIVER	c_p	0.1
	c_v	0.5
	c_β	0.5
	r_0	50
	$p_{\max}$	20 [m]

DYNAMICS

Descript.	Bounds
Acc.	$\lvert a\rvert \leq 1.2$ [m/s^2]
Ang. vel.	$\lvert w\rvert \leq 1.2$ [rad/s]
Steering	$\lvert\beta\rvert \leq 0.55$ [rad]
Speed	$v \in [0, 3.3]$ [m/s]

REFERENCES

[1] M. Maurer, J. C. Gerdes, B. Lenz, and H. Winner, Eds., *Autonomous Driving: Technical, Legal and Social Aspects*. Heidelberg: Springer, 2016.

[2] M. Bojarski, P. Yeres, A. Choromanska, K. Choromanski, B. Firner, L. Jackel, and U. Muller, "Explaining how a deep neural network trained with end-to-end learning steers a car," 2017.

[3] K. Ogata, *Modern Control Engineering*, 5th ed. Pearson, 2010.

[4] D. Kim, J. Kang, and K. Yi, "Control strategy for high-speed autonomous driving in structured road," *International IEEE Conference on Intelligent Transportation Systems*, pp. 186–191, 2011.

[5] F. Lin, Z. Lin, and X. Qiu, "LQR controller for car-like robot," in *35th Chinese Control Conference (CCC)*, 2016, pp. 2515–2518.

[6] N. Tavan, M. Tavan, and R. Hosseini, "An optimal integrated longitudinal and lateral dynamic controller development for vehicle path tracking," *Latin American Journal of Solids and Structures*, vol. 12, pp. 1006–1023, 2015.

[7] P. Falcone, F. Borrelli, J. Asgari, and D. Hrovat, "Low complexity MPC schemes for integrated vehicle dynamics control problems," *International Symposium on Advanced Vehicle Control*, 2008.

[8] L. Sommer, M. Rick, A. Folkers, and C. Büskens, "AO-Car: transfer of space technology to autonomous driving with the use of WORHP," in *Proceedings of the 7th International Conference on Astrodynamics Tools and Techniques*, 2018.

[9] M. Knauer and C. Büskens, "From WORHP to TransWORHP," in *Proceedings of the 5th International Conference on Astrodynamics Tools and Techniques*, 2012.

[10] S. Gu, E. Holly, T. Lillicrap, and S. Levine, "Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates," 2016.

[11] P. Abbeel, A. Coates, M. Quigley, and A. Y. Ng, "An application of reinforcement learning to aerobatic helicopter flight," in *Advances in Neural Information Processing Systems 19*, B. Schölkopf, J. C. Platt, and T. Hoffman, Eds. MIT Press, 2007, pp. 1–8.

[12] D. Isele, R. Rahimi, A. Cosgun, K. Subramanian, and K. Fujimura, "Navigating occluded intersections with autonomous vehicles using deep reinforcement learning," 2017.

[13] B. Mirchevska, M. Blum, L. Louis, J. Boedecker, and M. Werling, "Reinforcement learning for autonomous maneuvering in highway scenarios," in *Workshop Fahrerassistenzsysteme und automatisiertes Fahren*, 2017.

[14] A. E. Sallab, M. Abdou, E. Perot, and S. Yogamani, "Deep reinforcement learning framework for autonomous driving," *Electronic Imaging, Autonomous Vehicles and Machines*, pp. 70–76, 2017.

[15] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," 2015.

[16] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Proceedings of The 33rd International Conference on Machine Learning*, M. F. Balcan and K. Q. Weinberger, Eds., vol. 48, 2016, pp. 1928–1937.

[17] A. Kendall, J. Hawke, D. Janz, P. Mazur, D. Reda, J.-M. Allen, V.-D. Lam, A. Bewley, and A. Shah, "Learning to drive in a day," 2018.

[18] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017.

[19] R. S. Sutton and A. G. Barto, *Reinforcement learning - an introduction*, ser. Adaptive computation and machine learning. MIT Press, 2010.

[20] D. E. Rumelhart, G. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986.

[21] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[22] Z. Wang, V. Bapst, N. Heess, V. Mnih, R. Munos, K. Kavukcuoglu, and N. de Freitas, "Sample efficient actor-critic with experience replay," 2016.

[23] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *Proceedings of the 32nd International Conference on Machine Learning*, F. Bach and D. Blei, Eds., vol. 37, Lille, France, 2015, pp. 1889–1897.

[24] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.

[25] M. Rick, J. Clemens, L. Sommer, A. Folkers, K. Schill, and C. Büskens, "Autonomous driving based on nonlinear model predictive control and multi-sensor fusion," to appear in *Proceedings of the 10th IFAC Symposium on Intelligent Autonomous Vehicles*.

[26] P. Polack, F. Altch, B. d'Andra Novel, and A. de La Fortelle, "The kinematic bicycle model: A consistent model for planning feasible trajectories for autonomous vehicles?" in *2017 IEEE Intelligent Vehicles Symposium (IV)*, 2017, pp. 812–818.

[27] A. D. Luca, G. Oriolo, and C. Samson, "Feedback control of a nonholonomic car-like robot," in *Robot Motion Planning and Control*. Springer, 1998, ch. 4.

[28] S. Zagoruyko and N. Komodakis, "Paying more attention to attention: Improving the performance of convolutional neural networks via attention transfer," 2016.

[29] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014.

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:21:09 UTC from IEEE Xplore. Restrictions apply.